

✓ EE446 - TinyML - Assignment 2

✓ Importing Libraries

```
import pandas as pd
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns
from pylab import rcParams
from sklearn.model_selection import train_test_split
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.models import Model, load_model, Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Input, Dense, Activation
from tensorflow.keras.callbacks import ModelCheckpoint, TensorBoard
from tensorflow.keras import regularizers
from sklearn.preprocessing import LabelEncoder
import warnings
warnings.filterwarnings("ignore")
from sklearn.utils import class_weight
from sklearn.metrics import accuracy_score
import c_writer
from os.path import join
```

✓ Reading Data

```
# Reading the data and adding column header (feature) names
data = pd.read_csv("Network_anomaly_data.txt", sep=",", names=["duration", "protocoltype", "service",
"flag", "srcbytes", "dstbytes", "land", "wrongfragment", "urgent", "hot", "numfailedlogins", "loggedin", "numcompromi
"rootshell", "suattempted", "numroot", "numfilecreations", "numshells", "numaccessfiles", "numoutboundcmds", "ishost"]
```

```
"isquestlogin","count","srvcount","serrorrate", "srvserrorrate","rerrorrate","svrerrorrate","samesrvrate",
"diffsrvrate", "srvdiffhostrate","dsthostcount","dsthostsrvcount","dsthostsamesrvrate", "dsthostdiffsrvrate",
"dsthostsamesrcportrate","dsthostsrvdiffhostrate","dsthosterrorrate","dsthostsrverrorrate","dsthosterrorrate",
"dsthostsrverrorrate","attack", "lastflag"])
```

data # printing the dataframe

	duration	protocoltype	service	flag	srcbytes	dstbytes	land	wrongfragment	urgent	hot	...	dsthosts
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	
1	0	udp	other	SF	146	0	0	0	0	0	...	
2	0	tcp	private	S0	0	0	0	0	0	0	...	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	
4	0	tcp	http	SF	199	420	0	0	0	0	...	
...	...	...	...	...	...	...	...	...	...	...	...	
125968	0	tcp	private	S0	0	0	0	0	0	0	...	
125969	8	udp	private	SF	105	145	0	0	0	0	...	
125970	0	tcp	smtp	SF	2231	384	0	0	0	0	...	
125971	0	tcp	klogin	S0	0	0	0	0	0	0	...	
125972	0	tcp	ftp_data	SF	151	0	0	0	0	0	...	

125973 rows x 43 columns

Question 1: Data Preprocessing

- (a) Drop the 'land', 'urgent', 'numfailedlogins', 'numoutboundcmds' columns from the dataframe "data".

```
## INSERT YOUR CODE HERE ##
columns_to_drop = ['land', 'urgent', 'numfailedlogins', 'numoutboundcmds']
data = data.drop(columns=columns_to_drop)
```

- ✓ (b) Change any label that is not named normal to attack in the {'attack'} column of the dataframe data.

```
## INSERT YOUR CODE HERE ##
```

```
data['attack'] = data['attack'].apply(lambda x: 'normal' if x == 'normal' else 'attack')
```

```
data #<----- Print your modified dataframe "data"
```

	duration	protocoltype	service	flag	srcbytes	dstbytes	wrongfragment	hot	loggedin	numcompromised	.
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	
1	0	udp	other	SF	146	0	0	0	0	0	
2	0	tcp	private	S0	0	0	0	0	0	0	
3	0	tcp	http	SF	232	8153	0	0	1	0	
4	0	tcp	http	SF	199	420	0	0	1	0	
...	...	...	...	...	...	...	...	...	...	...	
125968	0	tcp	private	S0	0	0	0	0	0	0	
125969	8	udp	private	SF	105	145	0	0	0	0	
125970	0	tcp	smtp	SF	2231	384	0	0	1	0	
125971	0	tcp	klogin	S0	0	0	0	0	0	0	
125972	0	tcp	ftp_data	SF	151	0	0	0	1	0	

125973 rows × 39 columns

- ✓ (c) Use LabelEncoder() function from the sklearn.preprocessing library to convert non-numerical attributes in the {'protocoltype', 'service', 'flag', 'attack'} columns of the dataframe data to numerical values.

```
## INSERT YOUR CODE HERE ##
```

```
non_numerical_columns = ['protocoltype', 'service', 'flag', 'attack']
for col in non_numerical_columns:
```

```
le = LabelEncoder()
data[col] = le.fit_transform(data[col])
```

```
pd.DataFrame(data) #<----- Print your modified dataframe "data"
```

	duration	protocoltype	service	flag	srcbytes	dstbytes	wrongfragment	hot	loggedin	numcompromised	.
0	0	1	20	9	491	0	0	0	0	0	
1	0	2	44	9	146	0	0	0	0	0	
2	0	1	49	5	0	0	0	0	0	0	
3	0	1	24	9	232	8153	0	0	1	0	
4	0	1	24	9	199	420	0	0	1	0	
...	...	...	...	...	...	...	...	...	...	...	...
125968	0	1	49	5	0	0	0	0	0	0	
125969	8	2	49	9	105	145	0	0	0	0	
125970	0	1	54	9	2231	384	0	0	1	0	
125971	0	1	30	5	0	0	0	0	0	0	
125972	0	1	20	9	151	0	0	0	1	0	

125973 rows x 39 columns

✓ Feature Scaling and Train/Test Split

```
# All the features apart from Attack are what we are going to use to predict the attack status of the data
# attack = 1 (normal/not an attack) and attack = 0 (attack)
X = data.drop(['attack'],axis=1).to_numpy()

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit and transform the data
X_normalized = scaler.fit_transform(X)
```

```

X_normalized = scaler.fit_transform(X)

Y = data['attack'].to_numpy()
# Splitting X and y testing and training data
# we are taking 20% of the data for testing and 80% of the data for training
X_train, X_test, y_train, y_test = train_test_split(X_normalized, Y, test_size = 0.20)
# reshaping y test and train array
y_train = y_train.reshape(len(y_train),1)
y_test = y_test.reshape(len(y_test),1)

```

✓ Question 2: Dimensionality Reduction for Visualization

- ✓ (a) Use TSNE from the sklearn.manifold library to visualize the data in the test set (X_{test}) in 2D. In your figure, use color "red" to mark {attack} data points and color "blue" to mark {normal} data points.

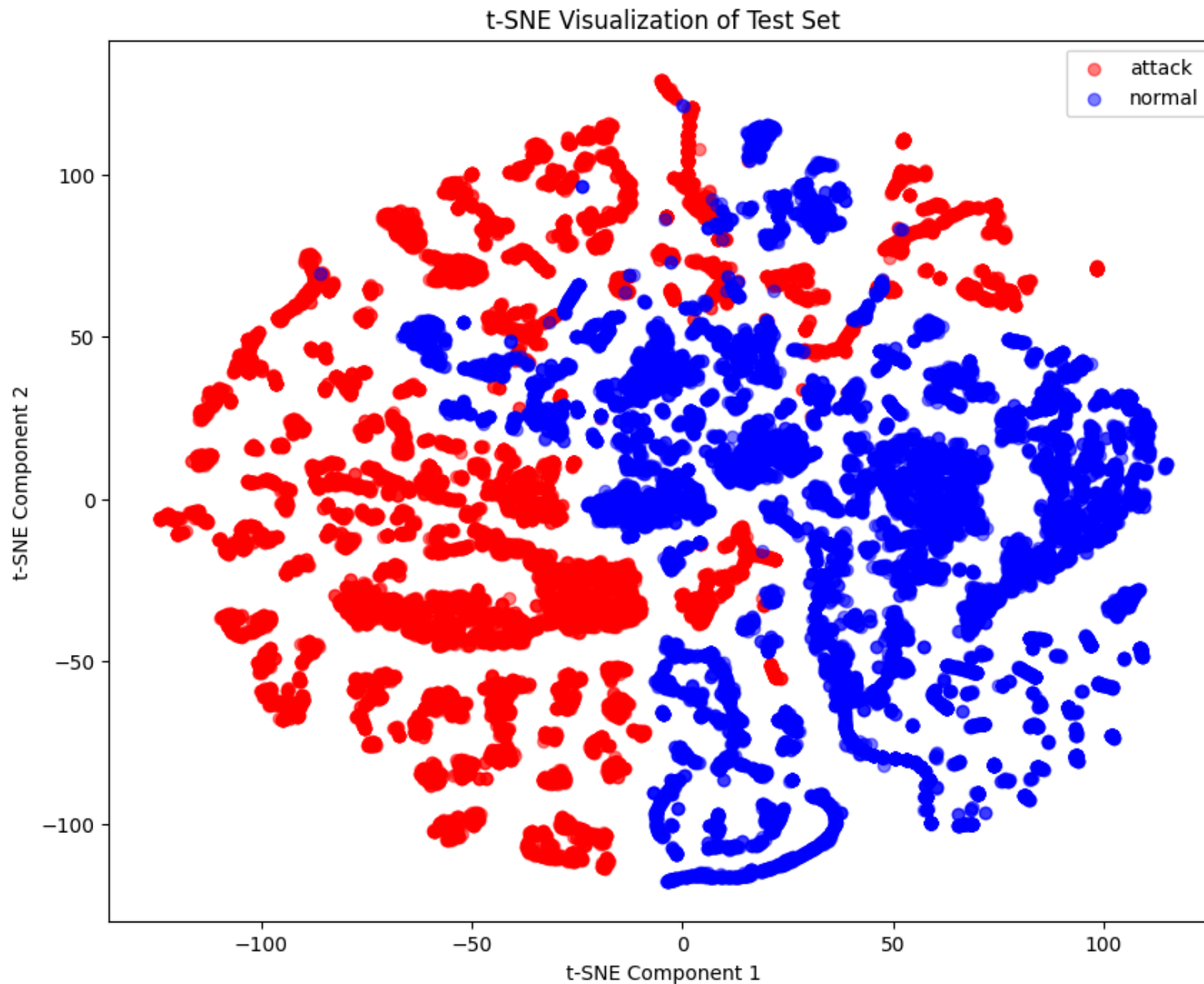
```

from sklearn.manifold import TSNE
## INSERT YOUR CODE HERE ##
tsne = TSNE(n_components=2, random_state=0)
X_test_2d = tsne.fit_transform(X_test)

color_map = {0: 'red', 1: 'blue'}    # attack=0 -> red, normal=1 -> blue
label_map = {0: 'attack', 1: 'normal'}

plt.figure(figsize=(10, 8))
for target in np.unique(y_test):
    mask = y_test.flatten() == target
    plt.scatter(X_test_2d[mask, 0], X_test_2d[mask, 1],
                color=color_map[target], label=label_map[target], alpha=0.5)
plt.title('t-SNE Visualization of Test Set')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.legend()
plt.show()

```

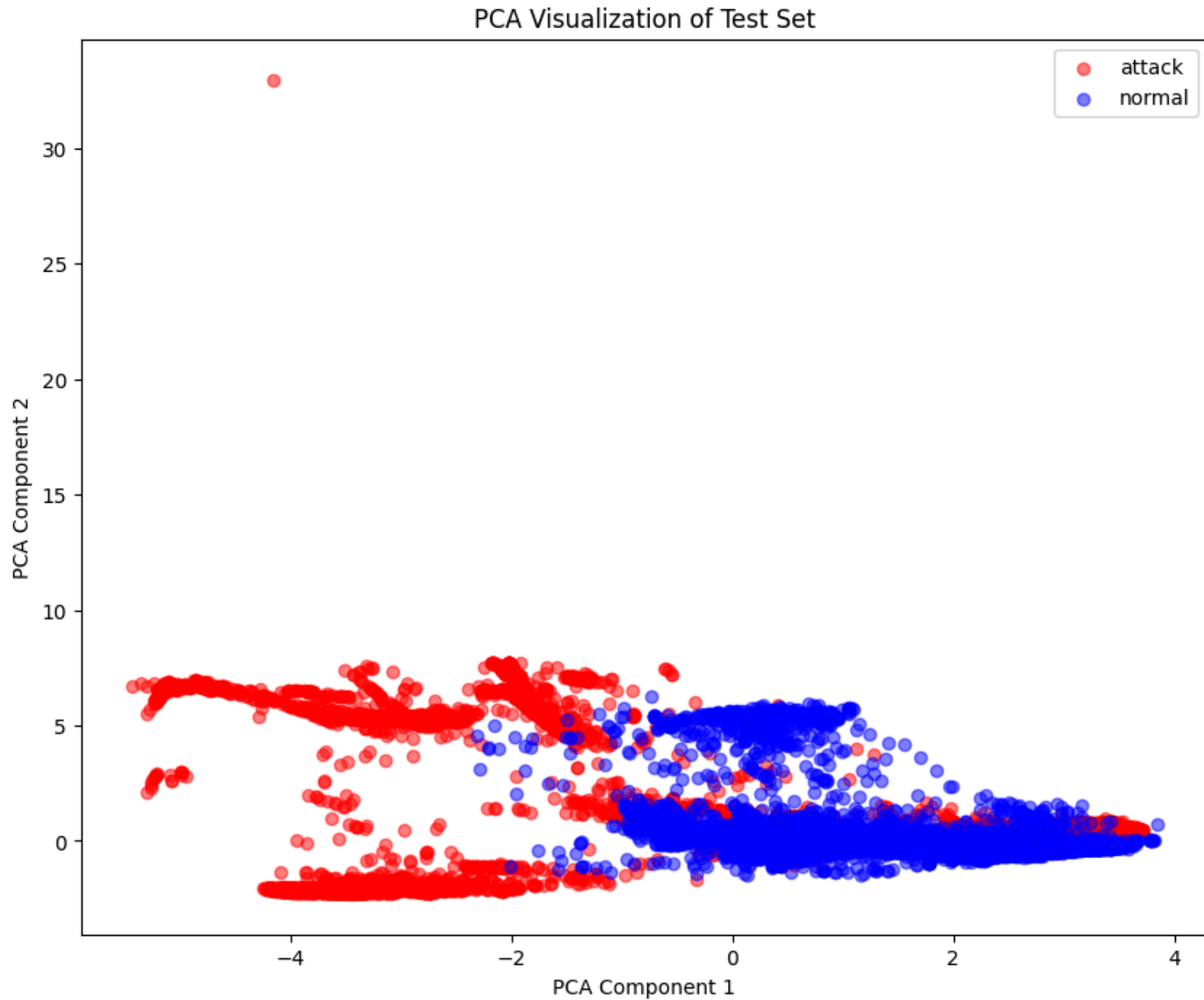


✓ (b) Use PCA from the `sklearn.decomposition` library to visualize the data in the test set (`X_test`) in 2D. In your figure, use color "red" to mark {attack} data points and color "blue" to mark {normal} data points.

```
from sklearn.decomposition import PCA
## INSERT YOUR CODE HERE ##
pca = PCA(n_components=2)
X_test_2d = pca.fit_transform(X_test)

color_map = {0: 'red', 1: 'blue'}    # attack=0 -> red, normal=1 -> blue
label_map = {0: 'attack', 1: 'normal'}

plt.figure(figsize=(10, 8))
for target in np.unique(y_test):
    mask = y_test.flatten() == target
    plt.scatter(X_test_2d[mask, 0], X_test_2d[mask, 1],
                color=color_map[target], label=label_map[target], alpha=0.5)
plt.title('PCA Visualization of Test Set')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend()
plt.show()
```

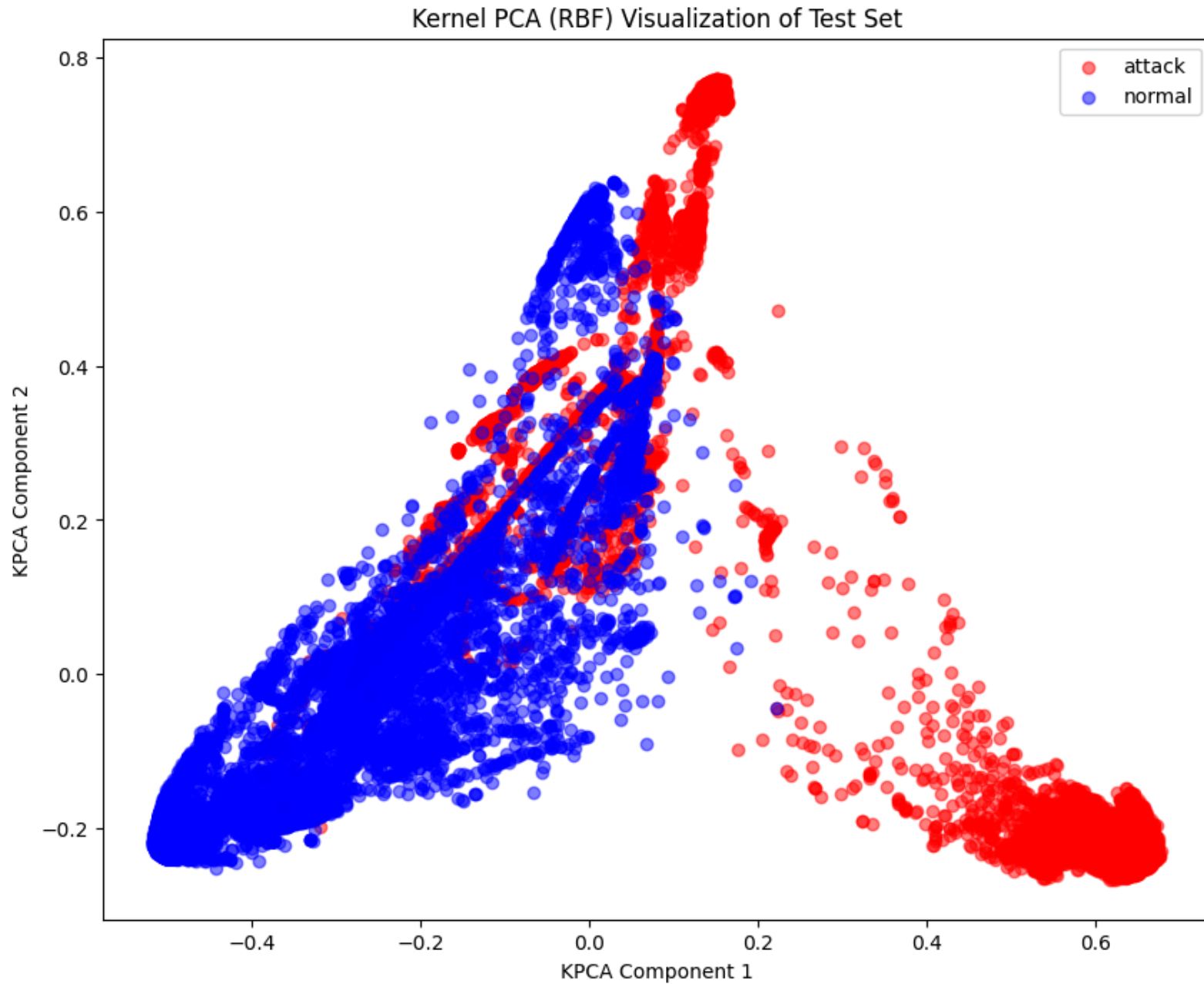


✓ (c) Use KernelPCA from the `sklearn.decomposition` library to visualize the data in the test set (X_{test}) in 2D. Use radial basis function (rbf) as the kernel. In your figure, use color "red" to mark {attack} data points and color "blue" to mark {normal} data points.


```
from sklearn.decomposition import KernelPCA
## INSERT YOUR CODE HERE ##
kpca = KernelPCA(n_components=2, kernel='rbf')
X_test_2d = kpca.fit_transform(X_test)

color_map = {0: 'red', 1: 'blue'}    # attack=0 -> red, normal=1 -> blue
label_map = {0: 'attack', 1: 'normal'}

plt.figure(figsize=(10, 8))
for target in np.unique(y_test):
    mask = y_test.flatten() == target
    plt.scatter(X_test_2d[mask, 0], X_test_2d[mask, 1],
                color=color_map[target], label=label_map[target], alpha=0.5)
plt.title('Kernel PCA (RBF) Visualization of Test Set')
plt.xlabel('KPCA Component 1')
plt.ylabel('KPCA Component 2')
plt.legend()
plt.show()
```



✓ Question 3: Implementing a DNN on the dataset

- ✓ (a) Implement a deep neural network (DNN) on the Network Anomaly Dataset. Ensure to include two neurons and softmax activation in the output layer of your DNN.

```
# Define the DNN model
base_model = Sequential([
    ## INSERT YOUR CODE HERE ##
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(2, activation='softmax')
])
```

- ✓ (b) Compile and train your DNN model on the training set (X_train). Denote the trained model as base_model.

```
## INSERT YOUR CODE HERE ##
base_model.compile(optimizer='adam',
                   loss='sparse_categorical_crossentropy',
                   metrics=['accuracy'])

history = base_model.fit(X_train, y_train,
                        epochs=10,
                        batch_size=32,
                        validation_split=0.2)
```

```
Epoch 1/10
2520/2520 ————— 9s 3ms/step - accuracy: 0.9931 - loss: 0.0240 - val_accuracy: 0.9952 - val_loss:
Epoch 2/10
2520/2520 ————— 8s 3ms/step - accuracy: 0.9968 - loss: 0.0087 - val_accuracy: 0.9955 - val_loss:
Epoch 3/10
2520/2520 ————— 8s 3ms/step - accuracy: 0.9976 - loss: 0.0069 - val_accuracy: 0.9963 - val_loss:
Epoch 4/10
2520/2520 ————— 9s 3ms/step - accuracy: 0.9980 - loss: 0.0052 - val_accuracy: 0.9970 - val_loss:
Epoch 5/10
2520/2520 ————— 9s 4ms/step - accuracy: 0.9981 - loss: 0.0049 - val_accuracy: 0.9971 - val_loss:
Epoch 6/10
2520/2520 ————— 7s 3ms/step - accuracy: 0.9984 - loss: 0.0042 - val_accuracy: 0.9966 - val_loss:
Epoch 7/10
2520/2520 ————— 8s 3ms/step - accuracy: 0.9985 - loss: 0.0040 - val_accuracy: 0.9974 - val_loss:
Epoch 8/10
2520/2520 ————— 6s 2ms/step - accuracy: 0.9989 - loss: 0.0033 - val_accuracy: 0.9980 - val_loss:
```

```
Epoch 9/10
2520/2520 ————— 11s 3ms/step - accuracy: 0.9989 - loss: 0.0032 - val_accuracy: 0.9982 - val_loss:
Epoch 10/10
2520/2520 ————— 8s 3ms/step - accuracy: 0.9988 - loss: 0.0045 - val_accuracy: 0.9980 - val_loss:
```

✓ (c) Evaluate the base_model on the test set (X_test) using classification_report and confusion_matrix from the sklearn.metrics library. Report these numbers in your .pdf writeup file using screenshots.

```
## INSERT YOUR CODE HERE ##
y_pred = base_model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_classes))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_classes))
```

```
788/788 ————— 1s 1ms/step
Confusion Matrix:
[[11811   15]
 [   19 13350]]

Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00     11826
     1           1.00        1.00        1.00     13369

 accuracy          1.00
 macro avg          1.00
weighted avg          1.00
```

```
# Save the original Keras model to HDF5 file
base_model.save('original_model.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`.
```

Question 4: Implementing Quantized Model

- ✓ (a) Implement Dynamic Range Quantization on the base_model. Designate the resulting quantized ML model as tflite_quant_model.

```
# Load the trained model
base_model = tf.keras.models.load_model('original_model.h5')

## INSERT YOUR CODE HERE ##
converter = tf.lite.TFLiteConverter.from_keras_model(base_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_quant_model = converter.convert()
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` Saved artifact at '/tmp/tmpnndtj603'. The following endpoints are available:

```
* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 38), dtype=tf.float32, name='input_layer')
Output Type:
  TensorSpec(shape=(None, 2), dtype=tf.float32, name=None)
Captures:
133659316041488: TensorSpec(shape=(), dtype=tf.resource, name=None)
133659316042640: TensorSpec(shape=(), dtype=tf.resource, name=None)
133659097930768: TensorSpec(shape=(), dtype=tf.resource, name=None)
133659097930384: TensorSpec(shape=(), dtype=tf.resource, name=None)
133659097929808: TensorSpec(shape=(), dtype=tf.resource, name=None)
133659097932688: TensorSpec(shape=(), dtype=tf.resource, name=None)
133659097930192: TensorSpec(shape=(), dtype=tf.resource, name=None)
133659097929616: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
import os

# Save the quantized model
with open('quantized_model.tflite', 'wb') as f:
    f.write(tflite_quant_model)

# Get the file sizes
original_model_size = os.path.getsize('original_model.h5')
quantized_model_size = os.path.getsize('quantized_model.tflite')
```

```
# Print the model sizes
print(f"Original model size: {original_model_size / 1024:.2f} KB")
print(f"Quantized model size: {quantized_model_size / 1024:.2f} KB")
```

```
Original model size: 216.25 KB
Quantized model size: 21.00 KB
```

✓

(b) Evaluate the `tflite_quant_model` on the test set (`X_test`) using `classification_report` and `confusion_matrix` from the `sklearn.metrics` library. Report these numbers in your .pdf writeup file using screenshots.

```
## INSERT YOUR CODE HERE ##
interpreter = tf.lite.Interpreter(model_content=tflite_quant_model)
interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

# Dynamic range quantization keeps float32 I/O, so we can feed samples directly.
y_pred_quant = []
for test_sample in X_test:
    inp = test_sample.reshape(1, -1).astype(np.float32)
    interpreter.set_tensor(input_details[0]['index'], inp)
    interpreter.invoke()
    output = interpreter.get_tensor(output_details[0]['index'])[0]
    y_pred_quant.append(np.argmax(output))

y_pred_quant_classes = np.array(y_pred_quant)

print("Confusion Matrix (Quantized Model):")
print(confusion_matrix(y_test, y_pred_quant_classes))
print("\nClassification Report (Quantized Model):")
print(classification_report(y_test, y_pred_quant_classes))
```

```
Confusion Matrix (Quantized Model):
[[11810   16]
 [   19 13350]]
```

```
Classification Report (Quantized Model):
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11826

1	1.00	1.00	1.00	13369
accuracy			1.00	25195
macro avg	1.00	1.00	1.00	25195
weighted avg	1.00	1.00	1.00	25195

✓ Converting tflite_model to C and create the header file

```
# c_writer is a py file in the same folder and has been imported at the beginning of the notebook
# Reference : https://github.com/ShawnHymel/tinymml-example-anomaly-detection/blob/master/utils/c_writer.py
# We use #04x to pad the output to 2 digits with a 0x prefix
hex_array = [format(val, '#04x') for val in tflite_quant_model]
# Calling function to convert an array into a C string (requires Numpy)
# create_array(np_array, var_type, var_name, line_limit=80, indent=4)
c_model = c_writer.create_array(np.array(hex_array), 'unsigned char', "network_model")
# Calling Function to create a header file with given C code as a string
header_str = c_writer.create_header(c_model, "network_model")
```

```
#Writing to the header file
with open('network_model.h', 'w') as file:
    file.write(header_str)
```

✓ Generating Samples for Inference on Arduino

```
# Converting a sample piece of the X test and y test data to C (for the purpose of ino code (arduino) to load a
# the sample and compare

Xtest = X_test[0:5,:]
print(c_writer.create_array(Xtest,"float","X_test"))
```

```

-0.007758502255079297, 0.005144515057050071, -0.00948042202040107,
-0.09507567152556495, 1.2356940323701657, -0.011663642603760032,
-0.036651869142258646, -0.024436507262009306, -0.01238515036740332,
-0.02618002418454278, -0.018609896340735923, -0.04122119759327531,
-0.0028174939213690777, -0.0975309439715147, -0.7257784945454897,
-0.17536713140262664, -0.6372092679572258, -0.6319290328885425,
-0.37436223991967527, -0.37443160310530493, 0.7712831058493207,
-0.349683030873482, 0.12576845924997587, 0.7343425609306344,
1.2587542737799418, 1.0664013456654926, -0.43907816809041417,
-0.4801968475158174, -0.28910340026287856, -0.6395319051152512,
-0.6248707997445304, -0.35501418179422695, -0.34508401858082666,
0.6528228780141483, -0.11024922321249885, -0.12470615670462065,
-0.6867852074966913, 0.7511112872365361, -0.007759004113055812,
-0.004918644383724874, -0.08948642202040107, -0.09507567152556495,
1.2356940323701657, -0.011663642603760032, -0.036651869142258646,
-0.024436507262009306, -0.01238515036740332, -0.02618002418454278,
-0.018609896340735923, -0.04122119759327531, -0.0028174939213690777,
-0.0975309439715147, -0.7170454919159579, -0.35434285002584825,
-0.6372092679572258, -0.6319290328885425, -0.37436223991967527,
-0.37443160310530493, 0.7712831058493207, -0.349683030873482,
-0.37455970440553465, 0.7343425609306344, -0.7104911462721373,
-0.8269154567466074, 2.7368516704978805, 0.005246480673521398,
-0.28910340026287856, -0.6395319051152512, -0.6248707997445304,
-0.38763462350750655, -0.3763870260680415, 0.6528228780141483,
-0.10986527309120776, -0.12470615670462065, 1.3931828317609012,
0.7511112872365361, -0.007017475714957735, -0.00483658041402168,
-0.08948642202040107, -0.09507567152556495, 1.2356940323701657,
-0.011663642603760032, -0.036651869142258646, -0.024436507262009306,
-0.01238515036740332, -0.02618002418454278, -0.018609896340735923,
-0.04122119759327531 -0.0028174939213690777 -0.0975309439715147

```